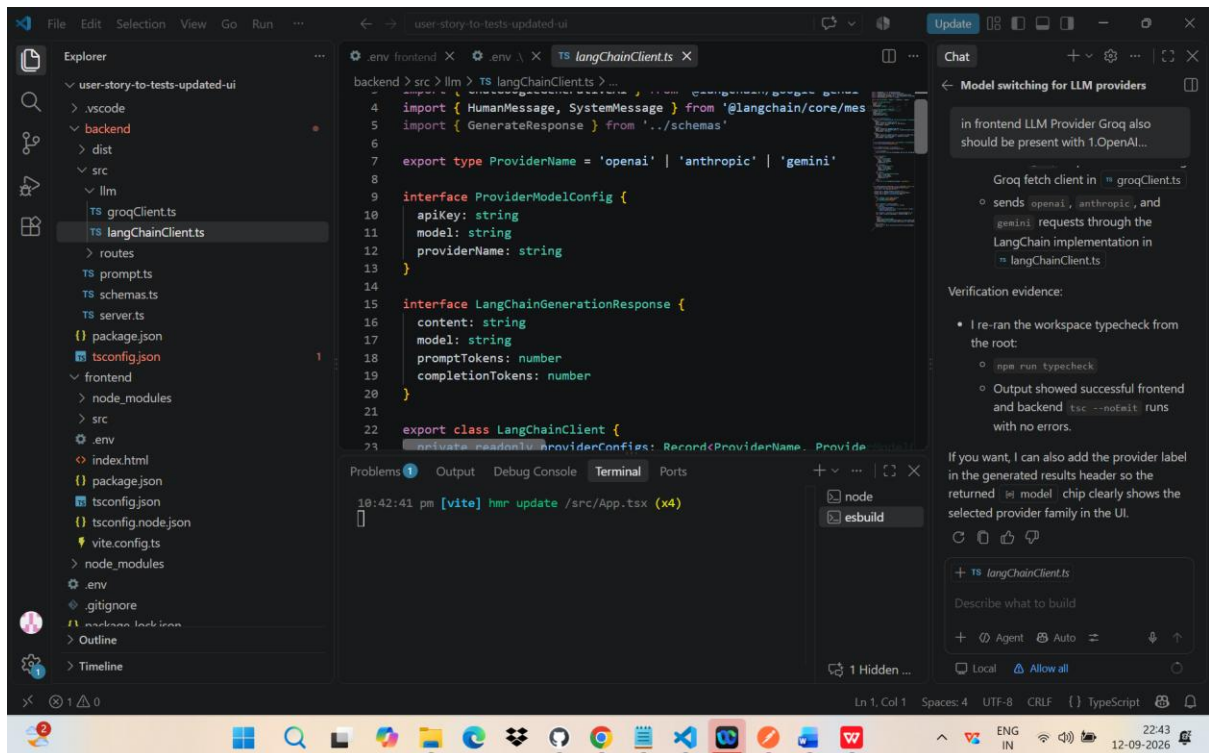


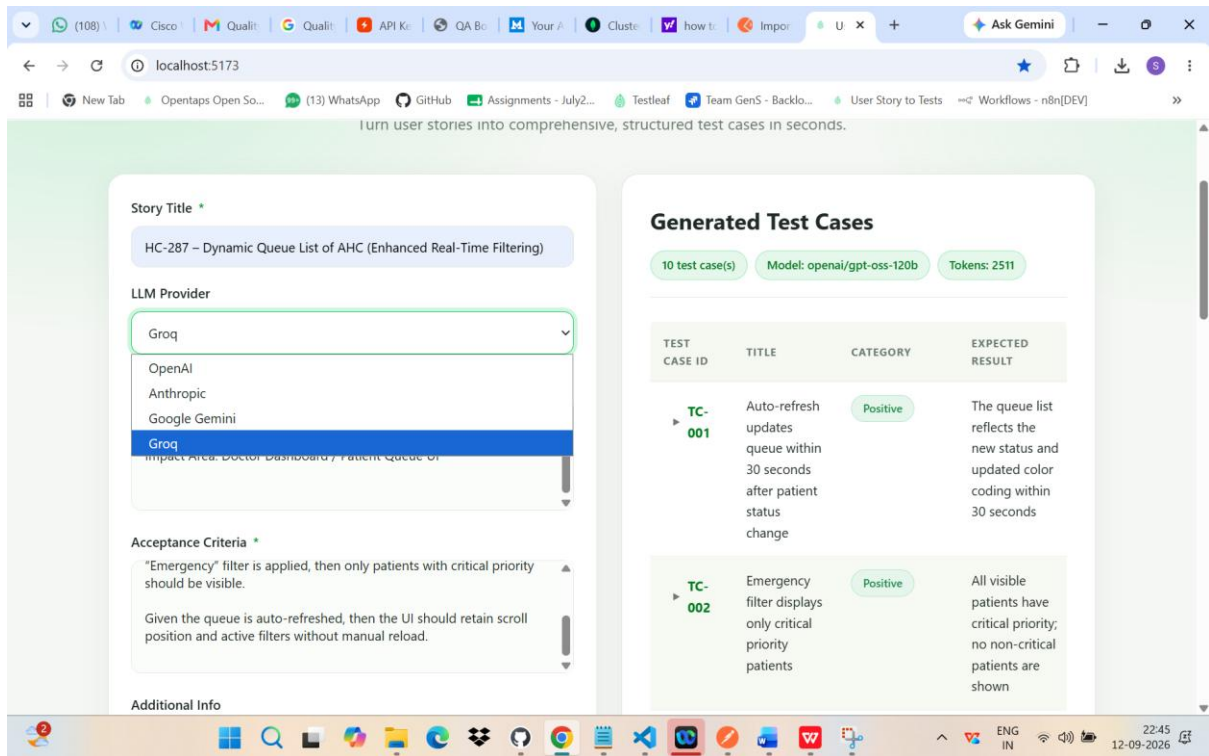
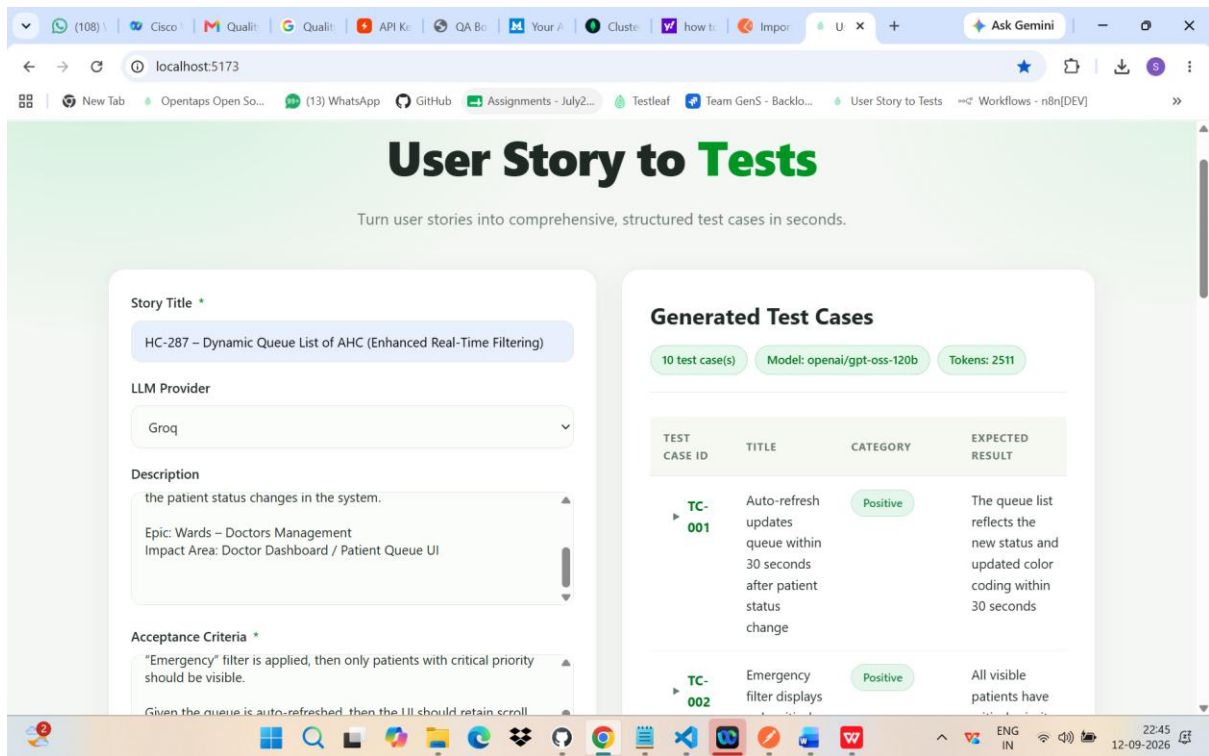
Home Assignment 2: LLM Model Switching

Use the user-story-to-tests-updated-ui application and implement a model-switching option for the following LLM providers using Langchain agent framework:

- 1.OpenAI
- 2.Anthropic
- 3.Google Gemini

Users should be able to select any provider from the UI and generate test cases using the selected model.





```
import { ChatOpenAI } from '@langchain/openai'
```

```
import { ChatAnthropic } from '@langchain/anthropic'
```

```

import { ChatGoogleGenerativeAI } from '@langchain/google-genai'
import { HumanMessage, SystemMessage } from '@langchain/core/messages'
import { GenerateResponse } from '../schemas'

export type ProviderName = 'openai' | 'anthropic' | 'gemini'

interface ProviderModelConfig {
  apiKey: string
  model: string
  providerName: string
}

interface LangChainGenerationResponse {
  content: string
  model: string
  promptTokens: number
  completionTokens: number
}

export class LangChainClient {
  private readonly providerConfigs: Record<ProviderName, ProviderModelConfig> = {
    openai: {
      apiKey: process.env.OPENAI_API_KEY || '',
      model: process.env.OPENAI_MODEL || 'gpt-4o-mini',
      providerName: 'OpenAI'
    },
    anthropic: {
      apiKey: process.env.ANTHROPIC_API_KEY || '',
      model: process.env.ANTHROPIC_MODEL || 'claude-3-5-sonnet-latest',
      providerName: 'Anthropic'
    },
  },

```

```

gemini: {
  apiKey: process.env.GEMINI_API_KEY || process.env.GOOGLE_API_KEY || "",
  model: process.env.GEMINI_MODEL || 'gemini-1.5-flash',
  providerName: 'Google Gemini'
}
}

```

```

private getProviderConfig(provider: ProviderName): ProviderModelConfig {
  return this.providerConfigs[provider]
}

```

```

private getModel(provider: ProviderName) {
  const config = this.getProviderConfig(provider)

```

```

    if (!config.apiKey) {
      throw new Error(`${config.providerName} API key not configured. Set ${provider === 'openai' ?
'OPENAI_API_KEY' : provider === 'anthropic' ? 'ANTHROPIC_API_KEY' : 'GEMINI_API_KEY'} in the
environment.`)
    }

```

```

switch (provider) {
  case 'openai':
    return new ChatOpenAI({
      model: config.model,
      apiKey: config.apiKey,
      temperature: 0.2
    })
  case 'anthropic':
    return new ChatAnthropic({
      model: config.model,
      apiKey: config.apiKey,
      temperature: 0.2

```

```

    })
    case 'gemini':
        return new ChatGoogleGenerativeAI({
            model: config.model,
            apiKey: config.apiKey,
            temperature: 0.2
        })
    default:
        throw new Error(`Unsupported provider: ${provider}`)
    }
}

```

```

async generateTests(systemPrompt: string, userPrompt: string, provider: ProviderName = 'openai'):
Promise<LangChainGenerationResponse> {

```

```

    const providerConfig = this.getProviderConfig(provider)
    const model = this.getModel(provider)

```

```

    console.log('🚀 Making LangChain provider call:')

```

```

    console.log(`📌 Provider: ${providerConfig.providerName}`)

```

```

    console.log(`🤖 Model: ${providerConfig.model}`)

```

```

    try {
        const response = await model.invoke([
            new SystemMessage(systemPrompt),
            new HumanMessage(userPrompt)
        ])

```

```

        const rawContent = response.content
        const content = typeof rawContent === 'string'
            ? rawContent
            : Array.isArray(rawContent)

```

```
      ? rawContent.map((item: any) => typeof item === 'string' ? item : item?.text ||
JSON.stringify(item)).join('\n')
      : JSON.stringify(rawContent)
```

```
const metadata = (response as any)?.response_metadata ?? {}
const usage = metadata.tokenUsage ?? metadata.usage ?? {}
```

```
return {
  content,
  model: providerConfig.model,
  promptTokens: Number(usage.input_tokens ?? usage.prompt_tokens ?? usage.promptTokens ??
0),
  completionTokens: Number(usage.output_tokens ?? usage.completion_tokens ??
usage.completionTokens ?? 0)
}
} catch (error) {
  console.error(` ❌ Error calling ${providerConfig.providerName} via LangChain:`, error)
  throw error
}
}
}
```